

SECUREBANK

Rapport de Sprint Beta

Version corrigée — incluant les correctifs appliqués après revue de code

Programmation Sécurisée – DevSecOps

DIC2-SSI – École Supérieure Polytechnique de Dakar

Enseignant : Doudou FALL

Équipe de développement

Cheikh Ahmed Tidiane Dieng

Fatou Ndour

Khadidiatou Faye

Mamadou Kone Ndour

Dakar, Juin 2026

1. Introduction

Le Sprint Beta constitue la seconde itération de développement du projet SecureBank. Appuyé sur les fondations techniques et sécuritaires établies lors du Sprint Alpha, il a pour ambition d'enrichir l'application de fonctionnalités avancées couvrant trois domaines complémentaires : le renforcement de la sécurité par des mécanismes de protection supplémentaires, l'amélioration de l'expérience utilisateur par des fonctionnalités de gestion du compte, et le renforcement de la supervision administrative par des outils d'analyse et de contrôle.

Ce sprint intervient dans un contexte où l'application est déjà pleinement fonctionnelle sur ses flux principaux : inscription, authentification à double facteur, gestion des comptes bancaires, virements avec confirmation OTP et journal d'audit HMAC. Le Sprint Beta ne remet pas en cause ces acquis mais les consolide et les enrichit, en traitant notamment des menaces résiduelles identifiées lors de l'analyse des exigences de la Partie 2 mais non adressées lors du Sprint Alpha.

Le présent rapport est la version corrigée du rapport de Sprint Beta initial. Suite à une revue de code approfondie, cinq catégories d'écarts ont été identifiées entre le rapport initial et l'implémentation effective : une fonctionnalité manquante (déconnexion par inactivité), deux fonctionnalités partiellement implémentées (masquage du numéro de compte et affichage du motif dans l'historique), une redirection incorrecte (mot de passe expiré), et trois manquements aux exigences du cours (analyse statique, gestion des secrets, couverture de tests). Tous ces points ont été corrigés dans le livrable final et sont documentés dans ce rapport.

2. Objectifs du Sprint Beta

Le Sprint Beta a été défini autour de dix-sept fonctionnalités regroupées en trois axes stratégiques, arrêtés en concertation avec l'équipe en début de sprint.

2.1 Renforcement de la sécurité

Le Sprint Alpha avait posé les bases de l'authentification et du chiffrement. Plusieurs vecteurs d'attaque demeuraient cependant non adressés. Le Sprint Beta visait à les couvrir intégralement à travers cinq mécanismes complémentaires.

- Détection des connexions depuis une adresse IP non reconnue, avec envoi automatique d'une alerte email au client concerné.
- Expiration automatique de la session JWT côté frontend, avec déconnexion propre accompagnée d'un message explicite et redirection vers la page de connexion.
- Déconnexion automatique après dix minutes d'inactivité, distincte de l'expiration du token JWT.
- Verrouillage temporaire du compte après trois tentatives de connexion consécutives échouées, pour une durée de quinze minutes.
- Expiration obligatoire du mot de passe après quatre-vingt-dix jours pour les clients et trente jours pour les administrateurs, avec changement forcé à la prochaine connexion.

2.2 Amélioration de l'expérience utilisateur

Plusieurs fonctionnalités de confort manquaient dans le Sprint Alpha et rendaient l'application incomplète du point de vue d'un utilisateur final. Le Sprint Beta les a toutes intégrées.

- Récupération de mot de passe oublié par envoi d'un code de réinitialisation par email, valable trente minutes.
- Changement de mot de passe depuis le profil, protégé par la saisie de l'ancien mot de passe, avec indicateur visuel de force du nouveau mot de passe.
- Validation des règles de complexité du mot de passe à la fois côté frontend et côté backend : huit caractères minimums, au moins un chiffre, au moins un caractère spécial.
- Masquage partiel du numéro de compte et affichage du solde sur demande (bouton œil).
- Affichage des comptes DE et VERS dans l'historique des transactions, accompagné du motif du virement lorsqu'il a été renseigné.

- Email de confirmation automatique envoyé au client après chaque virement effectué avec succès.
- Dépôt d'argent sur ses propres comptes par le client lui-même, en plus du dépôt administrateur.
- Demande de suppression de compte à l'administrateur, avec notification email automatique.
- Confirmation explicite avant déconnexion pour éviter les déconnexions accidentelles.

2.3 Renforcement de la supervision administrative

L'administrateur disposait dans le Sprint Alpha d'un accès aux utilisateurs et au journal d'audit. Le Sprint Beta lui a apporté des outils de gestion et d'analyse plus complets.

- Tableau de bord de statistiques globales anonymisées : total des montants virés, nombre total de transactions, nombre de clients actifs, volume journalier des opérations. Ces données sont volontairement agrégées sans détail des parties impliquées, conformément au principe de minimisation des données.
- Filtres sur les transactions par statut, plage de dates et plage de montants.
- Export CSV des transactions filtrées pour analyse externe.
- Gestion des demandes de suppression de compte soumises par les clients, avec possibilité d'approuver ou de refuser chaque demande.
- Dépôt d'argent sur n'importe quel compte client depuis le tableau de bord admin.
- Suppression directe d'un compte par l'administrateur, bloquée si le solde est non nul.
- Liste des comptes et utilisateurs suspendus.

3. Fonctionnalités livrées

3.1 Sécurité — Verrouillage après tentatives échouées

Le service d'authentification a été enrichi d'un compteur de tentatives échouées persisté en base de données dans la table users via le champ failedLoginAttempts. À chaque tentative de connexion avec un mot de passe incorrect, ce compteur est incrémenté. Lorsqu'il atteint trois, le compte est verrouillé pour quinze minutes grâce au champ lockedUntil. À chaque démarrage d'une tentative de connexion, le système vérifie si le champ lockedUntil est renseigné et supérieur à l'heure courante. Si c'est le cas, un message indique au client le nombre de minutes restantes avant déverrouillage. Un email d'alerte est également envoyé au compte concerné lors du verrouillage.

Le compteur est automatiquement réinitialisé à zéro et le verrou levé lors d'une connexion réussie. Ce mécanisme protège contre les attaques par force brute tout en restant transparent pour les utilisateurs légitimes qui ne dépassent pas le seuil.

3.2 Sécurité — Détection d'adresse IP suspecte

Chaque connexion réussie enregistre l'adresse IP du client dans le champ knownIps de la table users, sous forme d'une liste CSV des trois dernières adresses utilisées. Lors de chaque connexion, l'adresse IP entrante est comparée à cette liste. Si elle n'y figure pas, un email d'alerte est envoyé au client l'informant d'une connexion depuis une nouvelle adresse IP, avec l'adresse en question et

l'invitation à contacter l'administrateur si la connexion n'est pas de son fait. La nouvelle adresse est ensuite ajoutée à la liste, et si celle-ci dépasse trois entrées, la plus ancienne est supprimée par rotation.

L'adresse IP est extraite du header X-Forwarded-For en priorité pour supporter les déploiements derrière un proxy, avec repli sur l'adresse directe de la requête.

3.3 Sécurité — Expiration du mot de passe

Un champ `passwordChangedAt` de type `LocalDateTime` a été ajouté à l'entité `User`. Il est initialisé à la date de création du compte et mis à jour à chaque changement de mot de passe. Une méthode helper `isPasswordExpired()` calcule si la durée depuis le dernier changement dépasse le seuil défini par le rôle : quatre-vingt-dix jours pour `ROLE_CLIENT`, trente jours pour `ROLE_ADMIN`.

Si le mot de passe est expiré lors d'une connexion réussie, au lieu de générer un OTP, le serveur retourne un flag `passwordExpired : true` dans la réponse. Côté frontend, ce flag est intercepté dans `LoginPage.jsx` et redirige l'utilisateur vers la page de changement de mot de passe obligatoire. L'utilisateur ne peut accéder à aucune autre fonctionnalité tant qu'il n'a pas défini un nouveau mot de passe conforme aux règles de complexité.

3.4 Sécurité — Expiration et déconnexion automatique de session (CORRIGÉ)

Deux mécanismes complémentaires ont été implémentés côté frontend dans `App.jsx`. Le premier est une vérification périodique toutes les trente secondes de la date d'expiration du token JWT stocké en `localStorage`. Lorsque le token est expiré, l'état d'authentification est réinitialisé et l'utilisateur est redirigé vers la page de connexion avec un message explicite. Le second mécanisme est la suppression automatique du token au démarrage de l'application si celui-ci est déjà expiré, évitant ainsi une session fantôme persistante entre deux sessions de navigateur.

Le mécanisme de déconnexion par inactivité, absent du code initial, a été ajouté lors de la revue de code. Il est mis en œuvre via un `useRef` contenant un `setTimeout` de 600 000 ms (dix minutes), réinitialisé par un `useEffect` qui écoute six événements utilisateur : `mousemove`, `mousedown`, `keydown`, `touchstart`, `scroll` et `click`. Ces écouteurs sont ajoutés à la connexion et retirés à la déconnexion pour éviter les fuites mémoire.

3.5 Confort — Validation de la complexité du mot de passe

Une méthode `validatePassword()` a été ajoutée dans `AuthService.java`. Elle est invoquée à chaque opération impliquant un nouveau mot de passe : inscription, réinitialisation et changement. Elle vérifie trois règles cumulatives : longueur minimale de huit caractères, présence d'au moins un chiffre via une expression régulière, présence d'au moins un caractère spécial via une expression régulière couvrant les caractères les plus courants. En cas de violation, un message d'erreur explicite est retourné sans déclencher aucune opération côté base de données.

Côté frontend, un indicateur visuel de force du mot de passe a été intégré dans `ChangePasswordPage.jsx`. Il évalue le mot de passe sur quatre critères (longueur minimale, présence de chiffre, présence de caractère spécial, longueur supérieure à douze caractères) et affiche une barre de progression colorée ainsi qu'un libellé parmi Faible, Moyen, Fort et Très fort.

3.6 Confort — Récupération de mot de passe oublié

Un flux de réinitialisation en deux étapes a été implémenté. Lors de la première étape, l'utilisateur soumet son adresse email depuis la page `ForgotPasswordPage.jsx`. Le backend génère un code alphanumérique à huit caractères en majuscules via `UUID.randomUUID()`, le stocke hashé dans les champs `resetPasswordToken` et `resetPasswordTokenExpiry` (validité trente minutes), et l'envoie par email. La réponse ne révèle pas si l'email existe en base, retournant toujours le même message neutre pour éviter l'énumération d'emails.

Lors de la deuxième étape, l'utilisateur soumet son email, le code reçu et son nouveau mot de passe. Le backend vérifie la correspondance du code, sa non-expiration, et la conformité du nouveau mot de passe aux règles de complexité. En cas de succès, le nouveau mot de passe est haché et persisté, le token est invalidé, et le champ `passwordChangedAt` est mis à jour.

3.7 Confort — Changement de mot de passe depuis le profil

Un endpoint `POST /api/auth/change-password` a été ajouté, accessible uniquement aux utilisateurs authentifiés. Il exige la saisie de l'ancien mot de passe, vérifié via `passwordEncoder.matches()` avant toute modification. Il vérifie également que le nouveau mot de passe est différent de l'ancien et conforme aux règles de complexité. En cas de succès, un email de confirmation est envoyé au client. La page `ChangePasswordPage.jsx` intègre un indicateur de force visuel pour guider le choix du nouveau mot de passe.

3.8 Confort — Masquage partiel du numéro de compte (CORRIGÉ)

Lors de la revue de code, il a été constaté que `DashboardPage.jsx` affichait les numéros de compte en clair et les soldes en permanence, sans mécanisme de confidentialité. `DashboardPage.jsx` a été corrigé pour intégrer la fonction `maskAccountNumber()` qui remplace les chiffres du numéro de compte à l'exception des quatre derniers par des astérisques (ex : `SB-****1234`). Un bouton œil (toggle) permet à l'utilisateur d'afficher ou de masquer chaque solde individuellement. Le solde total dispose également d'un toggle indépendant.

3.9 Confort — Affichage DE / VERS et motif dans l'historique (CORRIGÉ)

`HistoryPage.jsx` affichait déjà les colonnes `sourceAccount` et `destinationAccount`, mais sans masquage des numéros et sans affichage du motif du virement. Le composant a été enrichi d'une colonne Détails permettant d'afficher le motif (champ label) dans une ligne expansible en cliquant sur l'icône chevron. Les numéros de compte sont masqués via la même fonction `maskNum()` que le tableau de bord. Un indicateur visuel (flèche $\uparrow$ / $\downarrow$) distingue les virements sortants des virements entrants. Un récapitulatif financier affiche le cumul des entrées et des sorties pour le compte sélectionné.

3.10 Confort — Redirection mot de passe expiré (CORRIGÉ)

`LoginPage.jsx` redirigait vers la page `forgot-password` lorsque le flag `passwordExpired` était reçu, ce qui était incorrect : la page de réinitialisation par email suppose que l'utilisateur ne connaît plus son mot de passe, alors que dans ce cas il vient de s'authentifier avec succès. La correction redirige désormais vers `change-password-forced`, route mappée sur `ChangePasswordPage` avec la prop `forced`, accessible sans token JWT complet. L'adresse email de l'utilisateur est transmise via `sessionStorage`.

3.11 Confort — Dépôt d'argent

Un endpoint POST `/api/accounts/{accountNumber}/deposit` a été ajouté dans `AccountController.java`, accessible aux clients authentifiés. Le service `AccountService.deposit()` vérifie que le compte appartient bien au client authentifié, que le compte est dans un état ACTIF, et que le montant est strictement positif. En cas de succès, le solde est mis à jour et l'opération est journalisée dans l'audit avec l'action DEPOSIT. Une page dédiée `DepositPage.jsx` permet au client de sélectionner un compte et de saisir un montant. Un endpoint symétrique POST `/api/admin/accounts/{number}/deposit` permet à l'administrateur d'effectuer des dépôts sur n'importe quel compte depuis le tableau de bord admin.

3.12 Confort — Demande et traitement de suppression de compte

Un workflow en deux étapes a été implémenté pour la suppression des comptes. Du côté client, un bouton Supprimer sur chaque compte ouvre une modale de confirmation. La demande est enregistrée dans la table `account_deletion_requests` avec le statut PENDING, et tous les administrateurs actifs sont notifiés par email. Du côté administrateur, un onglet Suppressions dans le tableau de bord admin liste toutes les demandes en attente. L'approbation est bloquée côté serveur si le solde du compte est supérieur à zéro.

3.13 Supervision admin — Statistiques anonymisées

Un endpoint GET `/api/admin/stats` retourne cinq indicateurs agrégés : nombre total de transactions complétées, montant total viré en FCFA, nombre de clients actifs, volume des transactions du jour et nombre de transactions du jour. Ces données sont calculées à la volée depuis les tables `transactions` et `users` sans exposer aucune information sur l'identité des parties. Cette approche respecte le principe de minimisation des données personnelles.

3.14 Supervision admin — Filtres et export CSV

L'endpoint GET `/api/admin/transactions` accepte cinq paramètres optionnels de filtrage : statut de la transaction, montant minimum, montant maximum, date de début et date de fin. Les transactions retournées sont anonymisées sans exposer les numéros de compte des parties. Un endpoint GET `/api/admin/transactions/export` génère un fichier CSV téléchargeable avec les mêmes paramètres de filtrage. Côté frontend, l'onglet Transactions de `AdminPage.jsx` présente un formulaire de filtres et un bouton d'export.

4. Outil d'analyse statique — SpotBugs + Find Security Bugs

L'exigence de recourir à un outil d'analyse statique (cours §Partie 3 et §Partie 4) était absente du livrable initial. Elle a été intégrée lors de la correction.

4.1 Outil retenu : SpotBugs avec le plugin Find Security Bugs

SpotBugs est l'outil d'analyse statique standard de l'écosystème Java. Il analyse le bytecode compilé et détecte les patterns de code problématiques. Le plugin Find Security Bugs (`findseccbugs`) est une extension spécialisée dans la détection de vulnérabilités de sécurité : injections SQL, XSS, chemins de fichiers non vérifiés, algorithmes cryptographiques faibles (MD5, DES, ECB...), gestion incorrecte des secrets et SSRF.

4.2 Intégration Maven

Le plugin `spotbugs-maven-plugin` a été ajouté au `pom.xml` avec le niveau d'effort Max et le seuil de signalement Medium. Le plugin Find Security Bugs est référencé comme plugin de SpotBugs. Un fichier `spotbugs-exclude.xml` documente les faux positifs connus liés à Spring Boot (proxies

Hibernate, injection @Autowired). L'analyse s'exécute avec la commande `mvn spotbugs:check` et le rapport XML est généré dans `target/spotbugs/`. L'interface graphique est disponible via `mvn spotbugs:gui`.

4.3 Résultats

L'analyse statique n'a pas détecté de vulnérabilité de sécurité critique dans le code produit par l'équipe. Les avertissements générés concernent principalement des patterns Hibernate (champs non initialisés dans les constructeurs d'entités JPA) et des proxies Spring, couverts par les exclusions documentées. Aucune alerte relative à un algorithme cryptographique faible, une injection, une gestion incorrecte des secrets ou un accès non sécurisé au système de fichiers n'a été détectée.

5. Gestion des secrets et des credentials

La revue de code a identifié que le fichier `application.yml` du livrable initial contenait en clair le mot de passe MySQL, le mot de passe d'application Gmail, la clé secrète JWT et la clé AES. Cette pratique est contraire aux bonnes pratiques DevSecOps et potentiellement dangereuse si le dépôt est rendu public.

5.1 Correction appliquée

Le fichier `application.yml` a été intégralement réécrit pour utiliser des variables d'environnement Spring Boot : `${DB_PASSWORD}`, `${MAIL_PASSWORD}`, `${JWT_SECRET}`, `${AES_KEY}`. Aucune valeur sensible ne figure plus dans le code source. Un fichier `.env.example` documente les variables à définir avec des valeurs d'exemple. Le fichier `.env` réel doit être ajouté au `.gitignore` et ne jamais être commité dans le dépôt de contrôle de source.

5.2 Procédure pour les évaluateurs

Pour faire fonctionner l'application, créer un fichier `.env` à la racine du projet backend en s'appuyant sur `.env.example` et en renseignant les valeurs réelles. Les variables d'environnement sont chargées automatiquement par Spring Boot au démarrage via la syntaxe `${VAR:valeur_par_defaut}`.

6. Modifications architecturales

6.1 Nouveaux champs dans l'entité User

L'entité User a été enrichie de six nouveaux champs pour supporter les fonctionnalités de sécurité du Sprint Beta. Le champ `failedLoginAttempts` de type `int` stocke le compteur de tentatives échouées. Le champ `lockedUntil` de type `LocalDateTime` stocke l'horodatage de fin de verrouillage. Le champ `passwordChangedAt` de type `LocalDateTime` stocke la date du dernier changement de mot de passe. Le champ `knownIps` de type `String` stocke la liste CSV des adresses IP connues. Les champs `resetPasswordToken` et `resetPasswordTokenExpiry` gèrent le cycle de vie des tokens de réinitialisation. Deux méthodes helper `isLocked()` et `isPasswordExpired()` encapsulent la logique de vérification.

6.2 Nouvelle entité AccountDeletionRequest

Une nouvelle entité JPA `AccountDeletionRequest` a été créée pour modéliser les demandes de suppression de compte. Elle contient le numéro de compte concerné, l'email du demandeur, le statut de la demande parmi `PENDING`, `APPROVED` et `REJECTED`, un commentaire optionnel de l'administrateur, et les horodatages de création et de traitement. Le repository associé

AccountDeletionRequestRepository expose des méthodes de recherche par email demandeur, par statut et par combinaison numéro de compte et statut.

6.3 Nouveau service EmailService

Un service EmailService dédié a été extrait pour centraliser l'ensemble des envois d'emails transactionnels. Il expose cinq méthodes spécialisées : envoi générique, alerte IP suspecte, email de réinitialisation de mot de passe, confirmation de virement et avertissement d'expiration prochaine du mot de passe. Chaque méthode produit un message formaté en texte plein avec le nom du destinataire, les informations contextuelles et une signature SecureBank.

6.4 Gestion des secrets (ajouté lors de la correction)

Toutes les valeurs sensibles sont désormais lues depuis des variables d'environnement via la syntaxe \${VAR} de Spring Boot. Aucun mot de passe, clé cryptographique ou secret ne figure dans les fichiers de configuration versionnés. Le fichier .env.example fournit un template documenté pour l'installation locale.

6.5 Analyse statique (ajouté lors de la correction)

SpotBugs et Find Security Bugs sont intégrés dans le cycle de build Maven via le plugin spotbugs-maven-plugin configuré pour s'exécuter lors de la phase verify. Le fichier spotbugs-exclude.xml documente et justifie les quatre faux positifs exclus liés aux patterns Spring Boot.

6.6 Base de données

Deux nouvelles tables ont été créées automatiquement par Hibernate lors du premier démarrage après les modifications du Sprint Beta : account_deletion_requests pour les demandes de suppression, et les nouvelles colonnes dans users (failed_login_attempts, locked_until, password_changed_at, known_ips, reset_password_token, reset_password_token_expiry).

6.7 Refonte du design frontend

Le fichier index.css a été entièrement réécrit pour produire une interface à l'ambiance bancaire professionnelle. Les polices ont été remplacées par Inter pour le corps de texte et Playfair Display pour les titres. La sidebar a été transformée en barre de navigation latérale vert foncé. De nouveaux composants CSS ont été introduits : bank-card, stat-card, tx-row et tabs.

7. Tests effectués

L'ensemble des fonctionnalités du Sprint Beta a été validé par des tests manuels sur l'interface frontend et des tests directs via curl sur l'API REST. La suite de tests automatisés a également été enrichie de 18 tests unitaires couvrant les exigences de sécurité et le module cryptographique.

7.1 Tests de sécurité

- ✓ Verrouillage après trois tentatives : validé. Après trois mauvais mots de passe consécutifs, le compte est verrouillé et un email est envoyé. Le message indique le nombre de minutes restantes.
- ✓ Déverrouillage automatique : validé. Après quinze minutes, la connexion est à nouveau possible.
- ✓ Détection IP suspecte : validé. La première connexion déclenche l'alerte email. Les connexions suivantes depuis la même IP ne déclenchent plus d'alerte.

- ✓ Expiration de mot de passe : validé. Un compte dont le passwordChangedAt est antérieur à la durée limite reçoit le flag passwordExpired et est redirigé vers le changement forcé (CORRIGÉ : vers change-password-forced, plus forgot-password).
- ✓ Validation de complexité : validé. Les mots de passe ne respectant pas les règles sont rejetés avec le message d'erreur approprié côté backend ET côté frontend.
- ✓ Expiration de session JWT : validé. Un token expiré est détecté au démarrage et supprimé automatiquement.
- ✓ Déconnexion par inactivité : validé (CORRIGÉ). La déconnexion intervient après dix minutes sans action utilisateur.

7.2 Tests des fonctionnalités utilisateur

- ✓ Mot de passe oublié : validé. Le code de réinitialisation est reçu par email, utilisable une seule fois dans les trente minutes.
- ✓ Changement de mot de passe : validé. L'ancien mot de passe incorrect est rejeté. Le nouveau doit être différent de l'ancien.
- ✓ Masquage du numéro de compte : validé (CORRIGÉ). Les 4 derniers chiffres sont visibles, le reste est masqué par des astérisques.
- ✓ Affichage du solde sur demande : validé (CORRIGÉ). Le bouton œil bascule l'affichage du solde pour chaque compte individuellement.
- ✓ Affichage DE/VERS et motif dans l'historique : validé (CORRIGÉ). Les numéros de compte sont masqués et le motif est accessible via le chevron.
- ✓ Dépôt client : validé. Le solde est mis à jour immédiatement. Un dépôt sur un compte ne lui appartenant pas est refusé.
- ✓ Dépôt sur montant zéro : validé. La requête est rejetée avec le message approprié.
- ✓ Demande de suppression avec solde positif : validé. La demande est acceptée et l'administrateur est notifié par email.
- ✓ Double demande sur le même compte : validé. La seconde demande est rejetée avec le message approprié.
- ✓ Confirmation avant déconnexion : validé. La boîte de dialogue apparaît et la déconnexion n'a lieu qu'après confirmation.

7.3 Tests de supervision admin

- ✓ Statistiques globales : validées. Les cinq indicateurs sont corrects et ne révèlent aucune information sur l'identité des parties.
- ✓ Filtres sur les transactions : validés. Les filtres par statut, montant et date fonctionnent indépendamment et en combinaison.
- ✓ Export CSV : validé. Le fichier téléchargé contient les transactions filtrées sans données personnelles.
- ✓ Approbation de suppression avec solde nul : validée. Le compte est supprimé et le statut de la demande passe à APPROVED.
- ✓ Approbation de suppression avec solde positif : validée. L'approbation est refusée avec le message indiquant le solde actuel.

- ✓ Refus d'une demande : validé. Le statut passe à REJECTED avec le commentaire de l'administrateur.
- ✓ Suppression directe par admin avec solde nul : validée.
- ✓ Dépôt admin : validé. Le solde du compte cible est mis à jour.

7.4 Tests automatisés — AuthServiceTest.java (12 tests)

Répartis en cinq classes imbriquées couvrant l'inscription (3 tests : email dupliqué, email nouveau, mot de passe faible), la validation de complexité (4 tests : trop court, sans chiffre, sans caractère spécial, valide), le verrouillage de compte (3 tests : compte verrouillé, verrou expiré, verrou actif), l'expiration du mot de passe (3 tests : client après 90 jours, admin après 30 jours, mot de passe récent) et la réinitialisation (2 tests : réponse neutre anti-énumération, token expiré).

7.5 Tests automatisés — AesEncryptorTest.java (6 tests)

Couvrent le module cryptographique AES-256-GCM : chiffrement/déchiffrement cohérent, IV aléatoire (deux chiffrements du même texte produisent des résultats différents), absence du texte clair dans le ciphertext, rejet d'un ciphertext altéré (intégrité du tag GCM), chiffrement d'une chaîne vide, et chiffrement d'un numéro de compte typique.

8. Difficultés rencontrées

8.1 Synchronisation frontend et backend

Le projet utilisant un frontend développé par un camarade séparément du backend, plusieurs incompatibilités ont été détectées lors de l'intégration. Les principales concernaient le format des réponses API (structure du champ data), la gestion des tokens JWT (extraction du rôle et de l'email depuis le payload plutôt que depuis le corps de la réponse) et la présence d'appels doubles inutiles sur le flux de virement. Ces problèmes ont été résolus en livrant des fichiers frontend corrigés et compatibles avec le backend existant.

8.2 Sérialisation JSON circulaire

Les entités JPA comportant des relations bidirectionnelles (User vers Account, Account vers Transaction) provoquaient des boucles de sérialisation JSON générant des réponses de plus de 250 000 caractères. La solution a consisté à annoter avec @JsonIgnore toutes les relations inverses dans les entités et à ajouter @JsonIgnoreProperties sur les entités concernées pour neutraliser les proxies Hibernate.

8.3 Cohérence de l'état knownIps

Le champ knownIps stockant une liste d'adresses IP sous forme de chaîne CSV présentait un risque de corruption en cas d'accès concurrent. Ce cas étant peu probable dans le contexte d'un projet pédagogique à utilisateurs réduits, il a été documenté comme limitation connue. Une implémentation robuste nécessiterait une table dédiée user_login_ips avec une contrainte d'unicité.

8.4 Port 8080 occupé entre les redémarrages

Le port 8080 restait régulièrement occupé par l'instance précédente du backend après un arrêt brutal via Ctrl+C. La solution systématique adoptée a été l'utilisation de la commande netstat combinée à taskkill avant chaque relance, intégrée dans un alias de terminal pour éviter la répétition manuelle.

8.5 Faux positifs SpotBugs liés à Spring Boot

L'intégration de SpotBugs a généré plusieurs avertissements liés aux patterns propres à Spring Boot : champs non initialisés dans les constructeurs d'entités JPA (initialisés par Hibernate), proxies de sécurité Spring, et détection d'un « hardcoded password » sur la clé JWT alors qu'elle est lue depuis une variable d'environnement. Ces faux positifs ont été documentés et exclus dans le fichier spotbugs-exclude.xml avec une justification explicite pour chacun.

9. Récapitulatif des corrections post-revue de code

Le tableau suivant résume les sept corrections apportées après la revue de code du Sprint Beta initial.

Problème identifié	Fichier(s) modifié(s)	Nature de la correction
Déconnexion inactivité 10 min absente du code	App.jsx	Ajout timer useRef + listeners sur 6 événements utilisateur
passwordExpired → redirige vers forgot-password	LoginPage.jsx	Redirection vers change-password-forced + email via sessionStorage
Numéro de compte affiché en clair	DashboardPage.jsx	Fonction maskAccountNumber() + bouton toggle solde par compte
Motif du virement non affiché	HistoryPage.jsx	Ligne expansible motif + indicateurs ↑/↓ + masquage numéros
Credentials en clair dans application.yml	application.yml + .env.example	Variables d'environnement \${VAR}, template .env.example documenté
Aucun outil d'analyse statique	pom.xml + spotbugs-exclude.xml	SpotBugs + Find Security Bugs, phase verify Maven
Suite de tests insuffisante (4 tests)	AuthServiceTest.java + AesEncryptorTest.java	18 tests : sécurité, crypto, verrouillage, expiration

10. Livrables du Sprint Beta

10.1 Fichiers backend créés ou modifiés

- ❖ User.java — six nouveaux champs Sprint Beta, deux méthodes helper isLocked() et isPasswordExpired()
- ❖ AuthService.java — logique complète de verrouillage, détection IP, expiration mot de passe, réinitialisation, changement
- ❖ AccountService.java — méthodes deposit() et requestDeletion()
- ❖ TransactionService.java — email de confirmation après virement
- ❖ EmailService.java — nouveau service transactionnel email
- ❖ AuthController.java — nouveaux endpoints forgot-password, reset-password, change-password
- ❖ AccountController.java — nouveaux endpoints deposit, request-deletion, deletion-requests

- ❖ AdminController.java — nouveaux endpoints stats, transactions filtrées, export CSV, dépôt admin, gestion des suppressions
- ❖ AccountDeletionRequest.java — nouvelle entité JPA
- ❖ AccountDeletionRequestRepository.java — nouveau repository
- ❖ SecurityConfig.java — nouveaux endpoints publics forgot-password et reset-password
- ❖ pom.xml — intégration SpotBugs + Find Security Bugs (CORRIGÉ)
- ❖ application.yml — externalisation de tous les secrets en variables d'environnement (CORRIGÉ)
- ❖ spotbugs-exclude.xml — filtres faux positifs documentés (CORRIGÉ)
- ❖ .env.example — template de configuration locale (CORRIGÉ)

10.2 Fichiers frontend créés ou modifiés

- ❖ index.css — refonte complète du design
- ❖ App.jsx — routage des nouvelles pages, vérification expiration JWT, confirmation déconnexion, déconnexion par inactivité (CORRIGÉ)
- ❖ Navbar.jsx — refonte avec classes sidebar
- ❖ LoginPage.jsx — lien mot de passe oublié, gestion flag passwordExpired avec redirection corrigée (CORRIGÉ)
- ❖ DashboardPage.jsx — masquage numéro de compte + toggle solde (CORRIGÉ)
- ❖ HistoryPage.jsx — affichage motif expansible + indicateurs ↑/↓ + masquage numéros (CORRIGÉ)
- ❖ AccountsPage.jsx — boutons déposer et supprimer, modale de confirmation
- ❖ AdminPage.jsx — onglets statistiques, filtres transactions, CSV, suppressions, dépôt admin
- ❖ ForgotPasswordPage.jsx — nouvelle page en deux étapes
- ❖ ChangePasswordPage.jsx — nouvelle page avec indicateur de force
- ❖ DepositPage.jsx — nouvelle page de dépôt client
- ❖ api.js — tous les nouveaux endpoints mappés

10.3 Fichiers de tests

- ❖ AuthServiceTest.java — 12 tests unitaires couvrant inscription, complexité, verrouillage, expiration, réinitialisation
- ❖ AesEncryptorTest.java — 6 tests unitaires couvrant le module cryptographique AES-256-GCM

10.4 Base de données

Deux nouvelles tables ont été créées automatiquement par Hibernate : `account_deletion_requests` pour les demandes de suppression, et les nouvelles colonnes dans `users` (`failed_login_attempts`, `locked_until`, `password_changed_at`, `known_ips`, `reset_password_token`, `reset_password_token_expiry`).

11. État final du projet après Sprint Beta

À l'issue du Sprint Beta (version corrigée), SecureBank est une application bancaire simplifiée fonctionnellement complète couvrant l'ensemble des cas d'usage définis dans le cahier des charges. Le système implémente une défense en profondeur cohérente avec les standards de sécurité bancaire à plusieurs niveaux.

- ✓ **Authentification** : BCrypt coût 12 + OTP email + JWT HS256 expiration 1h + verrouillage 3 tentatives + expiration mot de passe + déconnexion inactivité 10 min
- ✓ **Autorisation** : RBAC Spring Security 6 + vérification propriétaire côté service
- ✓ **Chiffrement** : AES-256-GCM au repos + TLS en transit
- ✓ **Intégrité** : Transactions ACID + chaînage HMAC-SHA256 du journal d'audit
- ✓ **Détection d'intrusion** : Alerte IP suspecte + journalisation de toutes les actions
- ✓ **Gestion du cycle de vie des comptes** : Dépôt, virement, demande de suppression, workflow d'approbation
- ✓ **Supervision** : Statistiques anonymisées + filtres + export CSV + gestion des demandes
- ✓ **Gestion des secrets** : Variables d'environnement, aucun secret commité dans le dépôt
- ✓ **Analyse statique** : SpotBugs + Find Security Bugs intégrés dans le build Maven
- ✓ **Interface** : Design bancaire professionnel responsive avec sidebar verte foncée

12. Conclusion

Le Sprint Beta a permis de transformer SecureBank d'un prototype fonctionnel en une application bancaire à la sécurité complète et à l'ergonomie soignée. Les dix-sept fonctionnalités planifiées ont toutes été livrées et testées avec succès. La revue de code post-livraison a permis d'identifier et de corriger sept écarts supplémentaires, dont certains concernaient des exigences explicites du cours non satisfaites dans le livrable initial.

Les corrections les plus significatives portent sur trois domaines distincts : la sécurité (déconnexion par inactivité effectivement opérationnelle, masquage des données sensibles côté client), le respect des exigences du cours (outil d'analyse statique SpotBugs intégré, suite de tests portée à 18 cas), et les bonnes pratiques DevSecOps (externalisation des credentials dans des variables d'environnement).

Les mécanismes de sécurité implémentés couvrent l'ensemble des menaces identifiées dans l'analyse de la Partie 2 : vol de mot de passe par force brute, connexion depuis une adresse IP non reconnue, session non révoquée, mot de passe vieux et compromis, et accès non autorisé aux comptes d'autres clients.

Le projet démontre la faisabilité d'une approche Security by Design dans le cadre d'un projet universitaire de taille réduite. Chaque fonctionnalité a été conçue avec la sécurité comme contrainte première plutôt que comme ajout ultérieur. Le journal d'audit avec chaînage HMAC, en particulier, illustre de manière concrète comment des techniques cryptographiques peuvent être appliquées à

des problèmes pratiques de sécurité informatique au-delà du simple chiffrement des communications.

Les deux sprints réalisés constituent une démonstration complète et cohérente des principes enseignés dans le cours de Programmation Sécurisée DevSecOps de l'ESP Dakar. Le projet est prêt pour la soutenance finale.